

Python for Computer Science and Data Science 2 (CSE 3652)

MINOR ASSIGNMENT-5: DEEP LEARNING

Question-1: Explain briefly Single layer perceptron and multilayer perceptron with architecture and illustrate the loss function associate with it.

A single layer perceptron (SLP) is the simplest form of a neural network, consisting of only an input layer and an output layer. There are no hidden layers. The architecture includes input nodes connected directly to output nodes via weights. Each output node computes a weighted sum of inputs, applies an activation function (typically a step function for binary classification), and produces a binary output. The SLP can only solve linearly separable problems, such as AND or OR logic gates, but cannot solve XOR.

A multilayer perceptron (MLP) extends the single layer perceptron by including one or more hidden layers between the input and output layers. Each hidden layer consists of neurons with nonlinear activation functions (e.g., sigmoid, tanh, ReLU). The architecture allows the network to learn complex, nonlinear decision boundaries. The output layer typically uses softmax for multi-class classification or a linear activation for regression.

The loss function for a perceptron (single layer) is often the perceptron criterion, which counts misclassifications, but more commonly for an MLP, we use cross-entropy loss for classification tasks and mean squared error (MSE) for regression. For binary classification with SLP, the loss is the number of misclassified samples. For MLP, the loss function (e.g., categorical cross-entropy) is minimized using backpropagation and gradient descent.

Question-2: How would you define the architecture of a simple feed forward ANN for classifying the Iris dataset? Write python code for the same.

For the Iris dataset (3 classes, 4 features), a simple feedforward ANN architecture can be defined as follows:

- Input layer: 4 neurons (one per feature)
- One hidden layer: 8 neurons with ReLU activation
- Output layer: 3 neurons with softmax activation (for multi-class classification)
- Loss function: Categorical cross-entropy
- Optimizer: Adam or SGD

Below is the Python code using TensorFlow/Keras.

```
In [1]: # Suppress TensorFlow warnings
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'

import tensorflow as tf
tf.get_logger().setLevel('ERROR')

# Import necessary Libraries
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.utils import to_categorical

# Load dataset
iris = load_iris()

X = iris.data
y = iris.target
```

```
# One-hot encode labels
y = to_categorical(y, num_classes=3)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Standardize features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build ANN model
model = Sequential([

    Input(shape=(4,)),

    Dense(8, activation='relu'),

    Dense(3, activation='softmax')
])

# Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train model
history = model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=5,
    validation_split=0.1,
    verbose=1
)

# Evaluate model
loss, accuracy = model.evaluate(
    X_test,
    y_test
)
```

```
Epoch 1/100
22/22 ————— 1s 13ms/step - accuracy: 0.3333 - loss: 1.1519 - val_accuracy: 0.2500 - v
al_loss: 1.0975
Epoch 2/100
22/22 ————— 0s 4ms/step - accuracy: 0.3611 - loss: 1.0764 - val_accuracy: 0.3333 - v
al_loss: 1.0553
Epoch 3/100
22/22 ————— 0s 4ms/step - accuracy: 0.4167 - loss: 1.0046 - val_accuracy: 0.3333 - v
al_loss: 1.0150
Epoch 4/100
22/22 ————— 0s 4ms/step - accuracy: 0.5000 - loss: 0.9351 - val_accuracy: 0.3333 - v
al_loss: 0.9751
Epoch 5/100
22/22 ————— 0s 5ms/step - accuracy: 0.6111 - loss: 0.8711 - val_accuracy: 0.5000 - v
al_loss: 0.9358
```

Epoch 6/100
22/22  0s 4ms/step - accuracy: 0.7222 - loss: 0.8076 - val_accuracy: 0.5000 - val_loss: 0.8991
Epoch 7/100
22/22  0s 4ms/step - accuracy: 0.7685 - loss: 0.7529 - val_accuracy: 0.5833 - val_loss: 0.8617
Epoch 8/100
22/22  0s 4ms/step - accuracy: 0.7870 - loss: 0.6985 - val_accuracy: 0.5833 - val_loss: 0.8276
Epoch 9/100
22/22  0s 4ms/step - accuracy: 0.8056 - loss: 0.6530 - val_accuracy: 0.5833 - val_loss: 0.7961
Epoch 10/100
22/22  0s 4ms/step - accuracy: 0.8056 - loss: 0.6128 - val_accuracy: 0.5833 - val_loss: 0.7679
Epoch 11/100
22/22  0s 4ms/step - accuracy: 0.8056 - loss: 0.5789 - val_accuracy: 0.6667 - val_loss: 0.7430
Epoch 12/100
22/22  0s 4ms/step - accuracy: 0.8148 - loss: 0.5497 - val_accuracy: 0.6667 - val_loss: 0.7213
Epoch 13/100
22/22  0s 4ms/step - accuracy: 0.8056 - loss: 0.5248 - val_accuracy: 0.6667 - val_loss: 0.7019
Epoch 14/100
22/22  0s 4ms/step - accuracy: 0.8056 - loss: 0.5037 - val_accuracy: 0.7500 - val_loss: 0.6853
Epoch 15/100
22/22  0s 4ms/step - accuracy: 0.8056 - loss: 0.4846 - val_accuracy: 0.7500 - val_loss: 0.6693
Epoch 16/100
22/22  0s 4ms/step - accuracy: 0.8056 - loss: 0.4684 - val_accuracy: 0.7500 - val_loss: 0.6561
Epoch 17/100
22/22  0s 5ms/step - accuracy: 0.8056 - loss: 0.4544 - val_accuracy: 0.7500 - val_loss: 0.6438
Epoch 18/100
22/22  0s 4ms/step - accuracy: 0.7963 - loss: 0.4416 - val_accuracy: 0.7500 - val_loss: 0.6323
Epoch 19/100
22/22  0s 4ms/step - accuracy: 0.7963 - loss: 0.4306 - val_accuracy: 0.7500 - val_loss: 0.6228
Epoch 20/100
22/22  0s 4ms/step - accuracy: 0.8148 - loss: 0.4194 - val_accuracy: 0.7500 - val_loss: 0.6133
Epoch 21/100
22/22  0s 4ms/step - accuracy: 0.8148 - loss: 0.4102 - val_accuracy: 0.7500 - val_loss: 0.6053
Epoch 22/100
22/22  0s 4ms/step - accuracy: 0.8148 - loss: 0.4012 - val_accuracy: 0.7500 - val_loss: 0.5974
Epoch 23/100
22/22  0s 4ms/step - accuracy: 0.8148 - loss: 0.3932 - val_accuracy: 0.7500 - val_loss: 0.5898
Epoch 24/100

```
22/22 ————— 0s 4ms/step - accuracy: 0.9352 - loss: 0.1584 - val_accuracy: 0.9167 - val_loss: 0.4075
Epoch 95/100
22/22 ————— 0s 4ms/step - accuracy: 0.9352 - loss: 0.1580 - val_accuracy: 0.9167 - val_loss: 0.3985
Epoch 96/100
22/22 ————— 0s 4ms/step - accuracy: 0.9352 - loss: 0.1556 - val_accuracy: 0.9167 - val_loss: 0.4063
Epoch 97/100
22/22 ————— 0s 4ms/step - accuracy: 0.9352 - loss: 0.1540 - val_accuracy: 0.9167 - val_loss: 0.4034
Epoch 98/100
22/22 ————— 0s 4ms/step - accuracy: 0.9444 - loss: 0.1527 - val_accuracy: 0.9167 - val_loss: 0.3979
Epoch 99/100
22/22 ————— 0s 4ms/step - accuracy: 0.9444 - loss: 0.1505 - val_accuracy: 0.9167 - val_loss: 0.3910
Epoch 100/100
22/22 ————— 0s 4ms/step - accuracy: 0.9444 - loss: 0.1491 - val_accuracy: 0.9167 - val_loss: 0.3898
1/1 ————— 0s 37ms/step - accuracy: 0.9667 - loss: 0.1248
Test Accuracy: 0.97
```

Question-3: How can you build and train a simple Artificial Neural Network (ANN) using the MNIST dataset to classify handwritten digits? Write python code for this.

A simple ANN for MNIST (28x28 grayscale images) flattens each image into a 784-dimensional vector.

Architecture:

- Input layer: 784 neurons
- Hidden layer: 128 neurons with ReLU
- Output layer: 10 neurons with softmax
- Loss: Categorical cross-entropy
- Optimizer: Adam

```
In [2]: # Suppress TensorFlow warnings
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'

import tensorflow as tf
tf.get_logger().setLevel('ERROR')

# Import Libraries
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Input
from tensorflow.keras.utils import to_categorical

# Load MNIST data
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize pixel values to 0-1
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# One-hot encode Labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Build model
model = Sequential([
    Input(shape=(28, 28)),
    Flatten(), # flattens 28x28 to 784
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
```

```
# Compile
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train
model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.2
)

# Evaluate
test_loss, test_acc = model.evaluate(x_test, y_test)
```

Epoch 1/5
1500/1500 ————— 3s 2ms/step - accuracy: 0.9140 - loss: 0.2970 - val_accuracy: 0.9563
- val_loss: 0.1556
Epoch 2/5
1500/1500 ————— 3s 2ms/step - accuracy: 0.9610 - loss: 0.1331 - val_accuracy: 0.9638
- val_loss: 0.1182
Epoch 3/5
1500/1500 ————— 3s 2ms/step - accuracy: 0.9738 - loss: 0.0907 - val_accuracy: 0.9708
- val_loss: 0.0980
Epoch 4/5
1500/1500 ————— 3s 2ms/step - accuracy: 0.9799 - loss: 0.0681 - val_accuracy: 0.9711
- val_loss: 0.1015
Epoch 5/5
1500/1500 ————— 3s 2ms/step - accuracy: 0.9844 - loss: 0.0525 - val_accuracy: 0.9728
- val_loss: 0.0924
313/313 ————— 0s 1ms/step - accuracy: 0.9733 - loss: 0.0858
Test accuracy: 0.9733

4. Find convolution , ReLu and Max Pooling with the following data Input image (4×4):

[[1, 2, 0, 1],
[3, 1, 2, 2],
[1, 0, 1, 3],
[2, 1, 2, 1]]

Filter/kernel (2×2):

[[1, 0],
[0, -1]]

Step-by-step solution:

Input image (4×4):

[1 2 0 1]
[3 1 2 2]
[1 0 1 3]
[2 1 2 1]

Kernel (2×2):

[1 0]
[0 -1]

Stride = 1, no padding

$$\begin{aligned}\text{Output size} &= (4 - 2 + 1) \times (4 - 2 + 1) \\ &= 3 \times 3\end{aligned}$$

Convolution result (element-wise multiplication and summation):

$$\text{Top-left: } (1 \times 1 + 2 \times 0 + 3 \times 0 + 1 \times (-1)) = 1 + 0 + 0 - 1 = 0$$

$$\text{Top-middle: } (2 \times 1 + 0 \times 0 + 1 \times 0 + 2 \times (-1)) = 2 + 0 + 0 - 2 = 0$$

$$\text{Top-right: } (0 \times 1 + 1 \times 0 + 2 \times 0 + 2 \times (-1)) = 0 + 0 + 0 - 2 = -2$$

$$\text{Middle-left: } (3 \times 1 + 1 \times 0 + 1 \times 0 + 0 \times (-1)) = 3 + 0 + 0 + 0 = 3$$

$$\text{Middle-middle: } (1 \times 1 + 2 \times 0 + 0 \times 0 + 1 \times (-1)) = 1 + 0 + 0 - 1 = 0$$

$$\text{Middle-right: } (2 \times 1 + 2 \times 0 + 1 \times 0 + 3 \times (-1)) = 2 + 0 + 0 - 3 = -1$$

$$\text{Bottom-left: } (1 \times 1 + 0 \times 0 + 2 \times 0 + 1 \times (-1)) = 1 + 0 + 0 - 1 = 0$$

$$\text{Bottom-middle: } (0 \times 1 + 1 \times 0 + 1 \times 0 + 2 \times (-1)) = 0 + 0 + 0 - 2 = -2$$

$$\text{Bottom-right: } (1 \times 1 + 3 \times 0 + 2 \times 0 + 1 \times (-1)) = 1 + 0 + 0 - 1 = 0$$

Convolution output (3×3):

$$\begin{bmatrix} 0 & 0 & -2 \\ 3 & 0 & -1 \\ 0 & -2 & 0 \end{bmatrix}$$

$$\text{ReLU activation: } f(x) = \max(0, x)$$

Applying ReLU:

$$\begin{bmatrix} 0 & 0 & 0 \\ 3 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Max Pooling:

$$\text{Pool size} = 2 \times 2$$

$$\text{Stride} = 1$$

Pooling regions:

Region 1:

$$\begin{bmatrix} 0 & 0 \\ 3 & 0 \end{bmatrix}$$

$$\max(0, 0, 3, 0) = 3$$

Region 2:

$$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\max(0, 0, 0, 0) = 0$$

Region 3:

$$\begin{bmatrix} 3 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\max(3, 0, 0, 0) = 3$$

Region 4:

$$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\max(0, 0, 0, 0) = 0$$

Max Pooling output:

$$\begin{bmatrix} 3 & 0 \\ 3 & 0 \end{bmatrix}$$

Final output after ReLU and Max Pooling:

$$\begin{bmatrix} 3 & 0 \\ 3 & 0 \end{bmatrix}$$

Question-5: How can you build a Convolutional Neural Network (CNN) with two convolutional layers and one fully connected hidden layer to classify handwritten digits from the MNIST dataset?

Below is the Python code using TensorFlow/Keras. Architecture:

- Conv2D layer 1: 32 filters, kernel size 3×3, ReLU activation
- MaxPooling2D layer 1: pool size 2×2
- Conv2D layer 2: 64 filters, kernel size 3×3, ReLU activation
- MaxPooling2D layer 2: pool size 2×2 Flatten layer
- Dense hidden layer: 128 neurons, ReLU activation
- Output layer: 10 neurons, softmax activation

```
In [3]: # Suppress TensorFlow warnings
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'

import tensorflow as tf
tf.get_logger().setLevel('ERROR')
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Input,
    Conv2D,
    MaxPooling2D,
    Flatten,
    Dense
)
from tensorflow.keras.utils import to_categorical

# Load data
(x_train, y_train), (x_test, y_test) = mnist.load_data()
# Reshape to (28, 28, 1) and normalize
x_train = x_train.reshape(-1, 28, 28, 1).astype('float32') / 255.0
x_test = x_test.reshape(-1, 28, 28, 1).astype('float32') / 255.0
# One-hot encode Labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
# Build CNN model
model = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(32, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
# Compile
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
# Train
model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)
# Evaluate
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test accuracy: {test_acc:.4f}")
```

Epoch 1/5
750/750  8s 8ms/step - accuracy: 0.9402 - loss: 0.1939 - val_accuracy: 0.9772 - val_loss: 0.0736
Epoch 2/5
750/750  6s 8ms/step - accuracy: 0.9826 - loss: 0.0545 - val_accuracy: 0.9830 - val_loss: 0.0570
Epoch 3/5
750/750  6s 8ms/step - accuracy: 0.9886 - loss: 0.0368 - val_accuracy: 0.9858 - val_loss: 0.0464
Epoch 4/5
750/750  6s 8ms/step - accuracy: 0.9915 - loss: 0.0281 - val_accuracy: 0.9873 - val_loss: 0.0410
Epoch 5/5
750/750  6s 8ms/step - accuracy: 0.9935 - loss: 0.0209 - val_accuracy: 0.9891 - val_loss: 0.0372
313/313  1s 2ms/step - accuracy: 0.9895 - loss: 0.0302
Test accuracy: 0.9895

Submitted By-

- Name- Amulya Shrivastava
- Regd No- 2341019456
- Section- 23412F3
- Serial No- 59
- Subject- PFCD2 Minor Assignment-05